

train_diffusion

January 21, 2026

1 Embryo Diffusion — Phase-Guided (silver F0) & Optional Classifier Guidance

This notebook trains a **conditional diffusion model** on the *silver* dataset (data/embryo_dataset_silver/F0) using the project's new structure. It conditions on the **morphokinetic phase** and supports **optional guidance** at sampling time using your pretrained **phase classifier** (external classifier guidance).

Project structure assumptions (edit paths in the next cell if needed):

```
thesis/  
  data/  
    embryo_dataset_bronze/          # raw frames (unused here)  
    embryo_dataset_silver/  
      <plane_id>/  
        <embryo_id>/  
          00001.jpg, 00002.jpg, ... # preprocessed frames (presence-filtered)  
          <embryo_id>_phases.csv    # copied unchanged  
  preprocessing/  
    embryo_presence_models/  
  classifier/  
    embryo_phase_models/  
  diffusion/  
    train_diffusion.ipynb           # ← this notebook (recommended location)  
    embryo_diffusion_models/        # checkpoints  
    diffusion_samples/              # generated samples
```

Notes - We **do not** run presence filtering here; *silver* data is already filtered by your preprocessing pipeline. - Training uses pairs $((x_t, x_{t+}))$ within each embryo sequence, with phases read from <embryo_id>_phases.csv. - The UNet is **phase-conditioned** (classifier-free guidance during training/sampling).

- Optionally, you can enable **external classifier guidance** at sampling using your phase classifier in classifier/embryo_phase_models/. If it fails to load, sampling gracefully falls back to classifier-free guidance.

```
[1]: # ==== CONFIG (edit if needed) ====  
from pathlib import Path  
import os
```

```

# DEBUG MODE: Set to True for quick testing with minimal data
DEBUG_MODE = True
VERBOSE = True # extra logging for dataset/split diagnostics

# If you open this notebook from thesis/diffusion/, PROJECT_ROOT points there
PROJECT_ROOT = Path.cwd().resolve()
if PROJECT_ROOT.name != 'diffusion':
    # try to infer if opened from repo root
    if (PROJECT_ROOT / 'diffusion').exists():
        PROJECT_ROOT = PROJECT_ROOT / 'diffusion'

DATA_ROOT = PROJECT_ROOT.parent / 'data'
SILVER_ROOT = DATA_ROOT / 'embryo_dataset_silver'
FOCAL_PLANES = ['F0', 'F-15', 'F-30', 'F-45', 'F15', 'F30', 'F45']

# Context window (k previous frames)
CONTEXT_K = 7 # Sharma-style fixed window (F=7)

# Time conditioning with absolute frame index embedding
USE_TIME_MAP = True
MAX_FRAME_INDEX = 500 # safe upper bound (dataset max ~450)
TIME_EMB_DIM = 16 # small, cheap

# Context dropout probabilities (oldest frames only; never drop t-1)
# 50%: full context, 25%: drop oldest 1, 15%: drop oldest 2, 10%: keep only t-1
CTX_DROP_P_FULL = 0.50
CTX_DROP_P_1 = 0.25
CTX_DROP_P_2 = 0.15
CTX_DROP_P_3 = 0.05

# Models
PHASE_MODEL_DIR = PROJECT_ROOT.parent / 'classifier' / 'embryo_phase_models'
PHASE_MODEL_PATH = None # autodetected if None

# Outputs
OUT_MODELS_DIR = PROJECT_ROOT / 'embryo_diffusion_models'
OUT_SAMPLES_DIR = PROJECT_ROOT / 'diffusion_samples'
CACHE_DIR = PROJECT_ROOT / 'cache'
for d in (OUT_MODELS_DIR, OUT_SAMPLES_DIR, CACHE_DIR):
    d.mkdir(parents=True, exist_ok=True)

# Training
IMG_SIZE = 128 # images are 500x500 in dataset; we center-crop &
               ↪resize
BATCH_SIZE = 32
LR = 1e-4 # Reduced from 2e-4 to prevent gradient explosion
EPOCHS = 100 if not DEBUG_MODE else 4 # Reduced epochs for debug

```

```

GRAD_CLIP = 1.0 # Gradient clipping to prevent NaN
NUM_WORKERS = 8
VAL_SPLIT = 0.1
TEST_SPLIT = 0.1
SPLIT_SEED = 42

# Diffusion
TIMESTEPS = 1000 # training timesteps
BETA_START, BETA_END = 1e-4, 2e-2
GUIDANCE_PROB = 0.9 # classifier-free guidance dropout during training
GUIDANCE_SCALE = 1.0 # CFG scale at sampling

# DDIM sampling/inversion
USE_DDIM_SAMPLING = True
DDIM_STEPS = 20
DDIM_ETA = 0.0

# Inversion training
INVERSION_TRAINING = True
INVERSION_LAMBDA = 0.1
INVERSION_STEPS = 10
INVERSION_EVERY_N = 16
INVERSION_MAX_BATCH = 8
USE_GRAD_CHECKPOINTING = False

# Sampling visualization
SAMPLE_PLANES = ['F0', 'F-15', 'F30']
SAMPLES_PER_PLANE = 3

# External classifier guidance (optional). If True and a classifier loads, use
↳ gradient guidance at sampling.
USE_CLASSIFIER_GUIDANCE = False
CLASSIFIER_GUIDANCE_SCALE = 0.5 # strength for external classifier guidance
↳ step
CLASSIFIER_INPUT_SIZE = 256 # if your classifier expects another size,
↳ change this
GUIDANCE_USE_SOFT_TARGETS = True
GUIDANCE_SMOOTH_SIGMA = 1.0 # soft-target smoothing over adjacent phases
GUIDANCE_STEP_EVERY = 2

# Data augmentation (applied identically to context stack + target)
AUGMENT = True
AUGMENT_HFLIP = 0.5
AUGMENT_VFLIP = 0.1
AUGMENT_MAX_ROT = 5.0
AUGMENT_BRIGHTNESS = 0.1
AUGMENT_CONTRAST = 0.1

```

```

# Loss weights
LPIPS_LOSS_WEIGHT = 0.0

# Validation metrics (SSIM/LPIPS)
COMPUTE_VAL_METRICS = True
VAL_METRIC_SAMPLES = 64 if not DEBUG_MODE else 4 # number of validation pairs
    ↪evaluated per epoch
VAL_METRIC_MAX_BATCHES = 4 # safety cap for validation metric computation

# Debug overrides
if DEBUG_MODE:
    NUM_WORKERS = 2
    BATCH_SIZE = 4
    DDIM_STEPS = 20
    INVERSION_STEPS = 8
    INVERSION_EVERY_N = 10
    INVERSION_MAX_BATCH = 2
    SAMPLE_PLANES = ['F0']
    SAMPLES_PER_PLANE = 1

print('Project root:', PROJECT_ROOT)
print('Silver root:', SILVER_ROOT)
print('Focal planes:', FOCAL_PLANES)
print(f'Context window k={CONTEXT_K}')
print(f'Splits: val={VAL_SPLIT:.2f}, test={TEST_SPLIT:.2f}, seed={SPLIT_SEED}')
print(f'DDIM: steps={DDIM_STEPS}, eta={DDIM_ETA}, enabled={USE_DDIM_SAMPLING}')
print(f'Inversion training: {INVERSION_TRAINING} (lambda={INVERSION_LAMBDA},
    ↪steps={INVERSION_STEPS}, every={INVERSION_EVERY_N},
    ↪max_batch={INVERSION_MAX_BATCH})')
print(f'Grad checkpointing: {USE_GRAD_CHECKPOINTING}')
if DEBUG_MODE:
    print(' DEBUG_MODE is ON - using reduced data for testing')
for plane in FOCAL_PLANES:
    plane_dir = SILVER_ROOT / plane
    assert plane_dir.exists(), f"Silver {plane} folder not found: {plane_dir}"

```

```

Project root: /mnt/c/Users/ioann/OneDrive/Documents/Projects/thesis/diffusion
Silver root:
/mnt/c/Users/ioann/OneDrive/Documents/Projects/thesis/data/embryo_dataset_silver
Focal planes: ['F0', 'F-15', 'F-30', 'F-45', 'F15', 'F30', 'F45']
Context window k=7
Splits: val=0.10, test=0.10, seed=42
DDIM: steps=20, eta=0.0, enabled=True
Inversion training: True (lambda=0.1, steps=8, every=10, max_batch=2)
Grad checkpointing: False
    DEBUG_MODE is ON - using reduced data for testing

```

```
[2]: # Imports & utils
import math, random, glob, re, os
import numpy as np
import pandas as pd
from PIL import Image
from tqdm import tqdm
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from torch.serialization import add_safe_globals
from torchvision.transforms import functional as TF

from lpips import LPIPS
from torchvision.models.efficientnet import EfficientNet

# Diffusers library for UNet and schedulers
from diffusers import UNet2DModel, DDPM Scheduler, DDIM Scheduler

# TQDM helper: force ETA/remaining to show consistently
TQDM_BAR_FORMAT = "{l_bar}{bar} {n_fmt}/{total_fmt} [{elapsed}<{remaining},
↳{rate_fmt}]"

def tqdm_bar(iterable, **kwargs):
    return tqdm(iterable, leave=True, bar_format=TQDM_BAR_FORMAT, **kwargs)
```

```
[3]: # Initialize seed and device (must be before using PHASE_LABELS in later cells)
def seed_everything(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

seed_everything(42)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Using device:', device)

PHASE_LABELS = [
    ↳['pPB2', 'pPNa', 'pPNf', 'p2', 'p3', 'p4', 'p5', 'p6', 'p7', 'p8', 'p9+', 'pM', 'pSB', 'pB', 'pEB', 'pHB']
NUM_PHASES = len(PHASE_LABELS)

# Plane ID mapping
PLANE_TO_ID = {plane: i for i, plane in enumerate(FOCAL_PLANES)}
```

```
ID_TO_PLANE = {i: plane for plane, i in PLANE_TO_ID.items()}
NUM_PLANES = len(FOCAL_PLANES)
```

Using device: cuda

```
[4]: # Build frame index from all focal planes: phase CSV lives next to frames
INDEX_CACHE = CACHE_DIR / 'silver_all_planes_frame_index.csv'

def build_index_from_silver():
    rows = []
    # For each focal plane
    for plane in FOCAL_PLANES:
        plane_root = SILVER_ROOT / plane
        embryo_ids = sorted([d.name for d in plane_root.iterdir() if d.
↪is_dir()])
        # For each embryo id included in focal plane
        for eid in tqdm(embryo_ids, desc=f'Indexing {plane}'):
            folder = plane_root / eid
            ann_path = folder / f'{eid}_phases.csv'
            if not ann_path.exists():
                continue
            # Use .jpeg files
            img_files = sorted(glob.glob(str(folder / '*.jpeg')))
            if not img_files:
                continue
            # Extract frame numbers from filenames (00037.jpeg + 37)
            frame_numbers = {}
            # For each frame image file
            for f in img_files:
                basename = os.path.basename(f)
                m = re.search(r'(\d+)\.jpeg$', basename)
                if m:
                    frame_num = int(m.group(1))
                    frame_numbers[frame_num] = f
            # read CSV (phase, start, end) with 1-based frame indices
            df = pd.read_csv(ann_path, header=None,
↪names=['phase', 'start', 'end'], sep='[,;\s]+', engine='python')
            # Phase must start with p, e.g., pPNA not tPNA
            def phase_to_id(ph: str):
                ph = str(ph).strip()
                ph = ph.replace('t', 'p', 1) if ph.startswith('t') else ph #
↪accept t* or p*
                return PHASE_LABELS.index(ph)
            label_spans = []
            for _, r in df.iterrows():
                try:
                    pid = phase_to_id(r['phase'])
```

```

        label_spans.append((int(r['start']), int(r['end']), pid))
    except Exception:
        pass
    # assign phase per frame using actual frame numbers from filenames
    for frame_num, path in frame_numbers.items():
        pid = None
        for s, e, p in label_spans:
            if s <= frame_num <= e: # frame_num is 1-based, matches CSV
                pid = p
                break
        if pid is None:
            continue
        rows.append({'embryo_id': eid, 'plane': plane, 'frame_num':
↪frame_num, 'path': path, 'phase_id': pid})
    return pd.DataFrame(rows)

if INDEX_CACHE.exists():
    index_df = pd.read_csv(INDEX_CACHE)
else:
    index_df = build_index_from_silver()
    index_df.to_csv(INDEX_CACHE, index=False)

print('Indexed frames (full):', len(index_df))
print('Planes:', sorted(index_df['plane'].unique().tolist()))
index_df.head()

```

```

Indexing F0: 100%|      | 704/704 [00:10<00:00, 65.53it/s]
Indexing F-15: 100%|    | 704/704 [00:08<00:00, 85.93it/s]
Indexing F-30: 100%|    | 704/704 [00:08<00:00, 87.63it/s]
Indexing F-45: 100%|    | 704/704 [00:07<00:00, 89.06it/s]
Indexing F15: 100%|     | 704/704 [00:07<00:00, 92.05it/s]
Indexing F30: 100%|     | 704/704 [00:07<00:00, 96.52it/s]
Indexing F45: 100%|     | 704/704 [00:07<00:00, 96.80it/s]

```

```

Indexed frames (full): 2028328
Planes: ['F-15', 'F-30', 'F-45', 'F0', 'F15', 'F30', 'F45']

```

```

[4]:  embryo_id plane  frame_num  \
0    AA83-7    F0           5
1    AA83-7    F0           6
2    AA83-7    F0           7
3    AA83-7    F0           8
4    AA83-7    F0           9

```

```

                                path  phase_id
0  /mnt/c/Users/ioann/OneDrive/Documents/Projects...    0
1  /mnt/c/Users/ioann/OneDrive/Documents/Projects...    0
2  /mnt/c/Users/ioann/OneDrive/Documents/Projects...    0

```

```
3 /mnt/c/Users/ioann/OneDrive/Documents/Projects... 0
4 /mnt/c/Users/ioann/OneDrive/Documents/Projects... 0
```

```
[5]: # Build embryo splits (no embryo appears in more than one split)
SPLIT_CACHE_STEM = (
    f'splits_embryo_phase_ctxk{CONTEXT_K}_'
    f'seed{SPLIT_SEED}_v{VAL_SPLIT}_t{TEST_SPLIT}'
    + ('_debug' if DEBUG_MODE else '')
)
TRAIN_SPLIT_CACHE = CACHE_DIR / f'{SPLIT_CACHE_STEM}_train.csv'
VAL_SPLIT_CACHE = CACHE_DIR / f'{SPLIT_CACHE_STEM}_val.csv'
TEST_SPLIT_CACHE = CACHE_DIR / f'{SPLIT_CACHE_STEM}_test.csv'

def stratified_split(ids: list[str], labels: dict[str, int], val_frac: float,
    ↪test_frac: float, seed: int):
    rng = np.random.RandomState(seed)
    train_ids, val_ids, test_ids = [], [], []
    by_label: dict[int, list[str]] = {}
    for eid in ids:
        by_label.setdefault(labels[eid], []).append(eid)
    for label, group in by_label.items():
        group = list(group)
        rng.shuffle(group)
        n = len(group)
        n_test = int(round(n * test_frac))
        n_val = int(round(n * val_frac))
        n_test = min(n_test, n)
        n_val = min(n_val, n - n_test)
        test_ids.extend(group[:n_test])
        val_ids.extend(group[n_test:n_test + n_val])
        train_ids.extend(group[n_test + n_val:])
    return set(train_ids), set(val_ids), set(test_ids)

# In DEBUG mode, sample only a few embryos
index_for_pairs = index_df.copy()
if DEBUG_MODE:
    unique_embryos = index_for_pairs['embryo_id'].unique()
    if len(unique_embryos) > 3:
        debug_embryos = np.random.choice(unique_embryos, 3, replace=False)
        index_for_pairs = index_for_pairs[index_for_pairs['embryo_id'].
    ↪isin(debug_embryos)]
        print(f'DEBUG_MODE: Using only {len(debug_embryos)} embryos for
    ↪testing')
    else:
        debug_embryos = unique_embryos

print('Indexed frames available:', len(index_for_pairs))
```

```

print('Planes in index:', sorted(index_for_pairs['plane'].unique().tolist()))

def _debug_split(ids: list[str]):
    ids = list(ids)
    rng = np.random.RandomState(SPLIT_SEED)
    rng.shuffle(ids)
    if len(ids) >= 3:
        return set(ids[2:]), {ids[1]}, {ids[0]}
    if len(ids) == 2:
        return {ids[1]}, {ids[0]}, set()
    if len(ids) == 1:
        return {ids[0]}, set(), set()
    return set(), set(), set()

if TRAIN_SPLIT_CACHE.exists() and VAL_SPLIT_CACHE.exists() and TEST_SPLIT_CACHE.
    exists():
        train_df = pd.read_csv(TRAIN_SPLIT_CACHE)
        val_df = pd.read_csv(VAL_SPLIT_CACHE)
        test_df = pd.read_csv(TEST_SPLIT_CACHE)
    else:
        embryo_phase = (
            index_for_pairs.groupby('embryo_id')['phase_id']
            .agg(lambda s: s.value_counts().idxmax())
            .to_dict()
        )
        embryo_ids = list(embryo_phase.keys())
        if DEBUG_MODE:
            train_ids, val_ids, test_ids = _debug_split(embryo_ids)
        else:
            train_ids, val_ids, test_ids = stratified_split(embryo_ids,
                embryo_phase, VAL_SPLIT, TEST_SPLIT, SPLIT_SEED)
            train_df = pd.DataFrame({'embryo_id': list(train_ids)})
            val_df = pd.DataFrame({'embryo_id': list(val_ids)})
            test_df = pd.DataFrame({'embryo_id': list(test_ids)})
            train_df.to_csv(TRAIN_SPLIT_CACHE, index=False)
            val_df.to_csv(VAL_SPLIT_CACHE, index=False)
            test_df.to_csv(TEST_SPLIT_CACHE, index=False)

train_ids = set(train_df['embryo_id'].tolist())
val_ids = set(val_df['embryo_id'].tolist())
test_ids = set(test_df['embryo_id'].tolist())

# DEBUG safeguard: ensure we have a non-empty val set for metrics/samples
if DEBUG_MODE:
    all_ids = sorted(set(index_for_pairs['embryo_id'].unique().tolist()))
    if len(val_ids) == 0 and len(all_ids) >= 2:
        train_ids, val_ids, test_ids = _debug_split(all_ids)

```

```

train_df = pd.DataFrame({'embryo_id': list(train_ids)})
val_df = pd.DataFrame({'embryo_id': list(val_ids)})
test_df = pd.DataFrame({'embryo_id': list(test_ids)})
train_df.to_csv(TRAIN_SPLIT_CACHE, index=False)
val_df.to_csv(VAL_SPLIT_CACHE, index=False)
test_df.to_csv(TEST_SPLIT_CACHE, index=False)

print(f'Splits cached: {TRAIN_SPLIT_CACHE.name}, {VAL_SPLIT_CACHE.name},
↳{TEST_SPLIT_CACHE.name}')
print(f"Train embryos: {len(train_ids)} | Val embryos: {len(val_ids)} | Test
↳embryos: {len(test_ids)}")

if VERBOSE:
    def _split_stats(name: str, ids: set[str]):
        df_split = index_for_pairs[index_for_pairs['embryo_id'].isin(ids)]
        frames = len(df_split)
        sequences = df_split.groupby(['embryo_id', 'plane']).ngroups
        planes = df_split['plane'].nunique()
        embryos = df_split['embryo_id'].nunique()
        print(f"[{name}] embryos={embryos} | planes={planes} |
↳sequences={sequences} | frames={frames}")
        _split_stats('train', train_ids)
        _split_stats('val', val_ids)
        _split_stats('test', test_ids)

```

```

DEBUG_MODE: Using only 3 embryos for testing
Indexed frames available: 5387
Planes in index: ['F-15', 'F-30', 'F-45', 'F0', 'F15', 'F30', 'F45']
Splits cached: splits_embryo_phase_ctxk7_seed42_v0.1_t0.1_debug_train.csv,
splits_embryo_phase_ctxk7_seed42_v0.1_t0.1_debug_val.csv,
splits_embryo_phase_ctxk7_seed42_v0.1_t0.1_debug_test.csv
Train embryos: 1 | Val embryos: 1 | Test embryos: 1
[train] embryos=1 | planes=7 | sequences=7 | frames=810
[val] embryos=1 | planes=7 | sequences=7 | frames=3108
[test] embryos=1 | planes=7 | sequences=7 | frames=1469

```

```

[6]: # Diffusion schedule using diffusers
from diffusers import DDPMPPipeline

scheduler = DDPMPPipeline(
    num_train_timesteps=TIMESTEPS,
    beta_start=BETA_START,
    beta_end=BETA_END,
    beta_schedule='scaled_linear',
    variance_type='fixed_small',
)

```

```

ddim_scheduler = DDIMScheduler(
    num_train_timesteps=TIMESTEPS,
    beta_start=BETA_START,
    beta_end=BETA_END,
    beta_schedule='scaled_linear',
    clip_sample=False,
    set_alpha_to_one=False,
)

# Pre-compute noise schedule for training
betas = scheduler.betas.to(device)
alphas = (1 - betas).to(device)
alphas_cumprod = torch.cumprod(alphas, dim=0).to(device)
sqrt_alphas_cumprod = torch.sqrt(alphas_cumprod)
sqrt_one_minus_alphas_cumprod = torch.sqrt(1.0 - alphas_cumprod)

# Compute posterior variance for DDPM sampling
# posterior_var_t = (1 - alpha_cumprod_{t-1}) / (1 - alpha_cumprod_t) * beta_t
alphas_cumprod_prev = torch.cat([torch.ones(1, device=device), alphas_cumprod[:
    ↪-1]])
posterior_var = (1 - alphas_cumprod_prev) / (1 - alphas_cumprod) * betas
posterior_var = torch.clamp(posterior_var, min=1e-20) # avoid log(0)

```

```

[7]: # Simplified diffusion model using diffusers UNet2DModel (context-conditioned)

class DiffusionCond(nn.Module):
    """Diffusion model conditioned on a context stack of k previous frames."""
    def __init__(self, context_k: int = CONTEXT_K, use_time_map: bool = ↪
    ↪USE_TIME_MAP):
        super().__init__()
        self.context_k = context_k
        self.use_time_map = use_time_map

        # Learned absolute frame index embedding (Sharma-style elapsed time ↪
    ↪proxy)
        if self.use_time_map:
            self.frame_emb = nn.Embedding(MAX_FRAME_INDEX + 1, TIME_EMB_DIM)
            self.frame_proj = nn.Linear(TIME_EMB_DIM, 1)
        else:
            self.frame_emb = None
            self.frame_proj = None

        # UNet2DModel from diffusers - lightweight attention at deeper layers
        # Input channels: 1 (noisy target) + k (context images) + optional time ↪
    ↪map
        in_ch = 1 + context_k + (1 if use_time_map else 0)
        self.unet = UNet2DModel(

```

```

        sample_size=IMG_SIZE
        , in_channels=in_ch
        , out_channels=1
        , layers_per_block=2
        , block_out_channels=(32, 64, 96, 96)
        , down_block_types=("DownBlock2D", "DownBlock2D", "DownBlock2D",
↪"DownBlock2D")
        , up_block_types=("UpBlock2D", "UpBlock2D", "UpBlock2D",
↪"UpBlock2D")
        , attention_head_dim=8
    )

    def build_time_map(self, frame_idx: torch.Tensor | int, h: int, w: int) ->
↪torch.Tensor:
        if not self.use_time_map:
            raise ValueError("build_time_map called but USE_TIME_MAP=False")
        if not torch.is_tensor(frame_idx):
            frame_idx = torch.tensor(frame_idx, device=next(self.parameters()).
↪device, dtype=torch.long)
        if frame_idx.dim() == 0:
            frame_idx = frame_idx.view(1)
        frame_idx = frame_idx.clamp(0, MAX_FRAME_INDEX).long()
        emb = self.frame_emb(frame_idx) # [B, TIME_EMB_DIM]
        scalar = self.frame_proj(emb) # [B, 1]
        return scalar.view(-1, 1, 1, 1).expand(-1, 1, h, w)

    def build_input(self, x_noisy: torch.Tensor, x_ctx: torch.Tensor, time_map:
↪torch.Tensor | None = None) -> torch.Tensor:
        """Assemble model input: [x_noisy, x_ctx, (optional) time_map]."""
        if self.use_time_map:
            if time_map is None:
                raise ValueError("time_map is required when USE_TIME_MAP=True")
            return torch.cat([x_noisy, x_ctx, time_map], dim=1)
        return torch.cat([x_noisy, x_ctx], dim=1)

    def forward(self, x_in, t):
        # x_in: [B, 1 + k (+1), H, W], t: [B]
        noise = self.unet(x_in, t).sample # [B, 1, H, W]
        return noise

    def q_sample(self, x0, t, noise=None):
        """Add noise to x0."""
        if noise is None:
            noise = torch.randn_like(x0)
        sqrt_alpha = sqrt_alphas_cumprod[t][:, None, None, None]
        sqrt_one_minus = sqrt_one_minus_alphas_cumprod[t][:, None, None, None]
        return sqrt_alpha * x0 + sqrt_one_minus * noise, noise

```

```

def p_losses(self, x_tgt, x_ctx, time_map: torch.Tensor | None = None):
    """Training loss components."""
    B = x_tgt.size(0)
    t = torch.randint(0, TIMESTEPS, (B,), device=x_tgt.device, dtype=torch.
↳long)
    x_noisy, noise = self.q_sample(x_tgt, t)
    x_in = self.build_input(x_noisy, x_ctx, time_map=time_map)
    pred = self.forward(x_in, t)
    mse = F.mse_loss(pred, noise)
    sqrt_alpha = sqrt_alphas_cumprod[t][:, None, None, None]
    sqrt_one_minus = sqrt_one_minus_alphas_cumprod[t][:, None, None, None]
    x0_pred = (x_noisy - sqrt_one_minus * pred) / (sqrt_alpha + 1e-8)
    return mse, x0_pred

```

```

[8]: def _preprocess_pil(img: Image.Image, size: int = IMG_SIZE) -> Image.Image:
    # center-crop square then resize
    w, h = img.size
    m = min(w, h)
    left = (w - m) // 2
    top = (h - m) // 2
    img = img.crop((left, top, left + m, top + m))
    if size is not None and (img.size[0] != size or img.size[1] != size):
        img = img.resize((size, size), Image.BICUBIC)
    return img

def load_pil(path: str, size: int = IMG_SIZE) -> Image.Image:
    img = Image.open(path).convert('L')
    return _preprocess_pil(img, size=size)

def pil_to_tensor(img: Image.Image) -> torch.Tensor:
    arr = np.array(img, dtype=np.float32) / 255.0
    arr = arr * 2 - 1 # [-1, 1]
    return torch.from_numpy(arr).unsqueeze(0)

def _augment_stack(ctx_list: list[Image.Image], tgt: Image.Image):
    """Apply identical random augmentation to every context frame + target."""
    if not AUGMENT:
        return ctx_list, tgt

    do_hflip = AUGMENT_HFLIP > 0 and random.random() < AUGMENT_HFLIP
    do_vflip = AUGMENT_VFLIP > 0 and random.random() < AUGMENT_VFLIP
    angle = random.uniform(-AUGMENT_MAX_ROT, AUGMENT_MAX_ROT) if
↳AUGMENT_MAX_ROT and AUGMENT_MAX_ROT > 0 else 0.0
    brightness = 1.0 + random.uniform(-AUGMENT_BRIGHTNESS, AUGMENT_BRIGHTNESS)
↳if AUGMENT_BRIGHTNESS and AUGMENT_BRIGHTNESS > 0 else None

```

```

        contrast = 1.0 + random.uniform(-AUGMENT_CONTRAST, AUGMENT_CONTRAST) if
↪AUGMENT_CONTRAST and AUGMENT_CONTRAST > 0 else None

    def _apply(img: Image.Image) -> Image.Image:
        if do_hflip:
            img = TF.hflip(img)
        if do_vflip:
            img = TF.vflip(img)
        if angle != 0.0:
            img = TF.rotate(img, angle, interpolation=TF.InterpolationMode.
↪BILINEAR)
        if brightness is not None:
            img = TF.adjust_brightness(img, brightness)
        if contrast is not None:
            img = TF.adjust_contrast(img, contrast)
        return img

    ctx_list = [_apply(img) for img in ctx_list]
    tgt = _apply(tgt)
    return ctx_list, tgt

def load_image(path: str, size: int = IMG_SIZE):
    img = load_pil(path, size=size)
    return pil_to_tensor(img)

def denorm(x):
    return (x.clamp(-1,1) + 1) / 2

def make_time_map(time_scalar: float, h: int, w: int, device=None) -> torch.
↪Tensor:
    """Legacy helper (unused when USE_TIME_MAP=True with frame embeddings)."""
    val = float(2 * time_scalar - 1)
    tmap = torch.full((1, h, w), val, dtype=torch.float32)
    return tmap.to(device) if device is not None else tmap

class PairDataset(Dataset):
    """Sharma-style windowed next-frame pairs (strict adjacency).
    Context frames [t-K+1..t] predict target frame t+1 (delta_t = 1).
    """
    def __init__(
        self,
        index_df: pd.DataFrame,
        embryo_ids: set[str],
        split_seed: int,
        split_name: str,
    ):

```

```

        self.df = index_df[index_df['embryo_id'].isin(set(embryo_ids))].copy().
↪reset_index(drop=True)
        self.split_name = split_name
        self.rng = np.random.RandomState(split_seed + {'train': 0, 'val': 1,
↪'test': 2}.get(split_name, 0))
        self.sequences: list[tuple[str, str]] = []
        self.sequences_by_plane: dict[str, list[tuple[str, str]]] = {}
        self.frames_by_seq: dict[tuple[str, str], list[int]] = {}
        self.path_by_seq: dict[tuple[str, str], dict[int, str]] = {}
        self.phase_by_seq: dict[tuple[str, str], dict[int, int]] = {}
        self.valid_t_ends_by_seq: dict[tuple[str, str], list[int]] = {}
        self.transition_t_ends_by_seq: dict[tuple[str, str], list[int]] = {}
        self.non_transition_t_ends_by_seq: dict[tuple[str, str], list[int]] = {}
        self.max_frame_by_seq: dict[tuple[str, str], int] = {}
        self.windows: list[dict] = []
        self.windows_by_plane: dict[str, list[dict]] = {}
        self.transition_windows_by_plane: dict[str, list[dict]] = {}
        self.non_transition_windows_by_plane: dict[str, list[dict]] = {}
        self._skipped_short = 0
        self._skipped_no_tgt = 0
        self._build_index()

def _build_index(self):
    for (eid, plane), g in self.df.groupby(['embryo_id', 'plane']):
        g = g.sort_values('frame_num').reset_index(drop=True)
        frame_nums = g['frame_num'].tolist()
        if len(frame_nums) <= CONTEXT_K:
            self._skipped_short += 1
            continue
        frame_set = set(frame_nums)
        path_by = dict(zip(frame_nums, g['path'].tolist()))
        phase_by = dict(zip(frame_nums, g['phase_id'].tolist()))
        max_frame = max(frame_nums)
        self.max_frame_by_seq[(eid, plane)] = max_frame

        valid_t_ends = []
        for t_end in frame_nums:
            # Require full context [t-K+1..t] and adjacent target t+1
            if (t_end + 1) not in frame_set:
                continue
            ok = True
            for f in range(t_end - CONTEXT_K + 1, t_end + 1):
                if f not in frame_set:
                    ok = False
                    break
            if ok:
                valid_t_ends.append(t_end)

```

```

        if not valid_t_ends:
            self._skipped_no_tgt += 1
            continue

        transition_t_ends = [t for t in valid_t_ends if phase_by.get(t) !=
↪phase_by.get(t + 1)]
        non_transition_t_ends = [t for t in valid_t_ends if phase_by.get(t)
↪== phase_by.get(t + 1)]

        seq = (eid, plane)
        self.sequences.append(seq)
        self.sequences_by_plane.setdefault(plane, []).append(seq)
        self.frames_by_seq[seq] = frame_nums
        self.path_by_seq[seq] = path_by
        self.phase_by_seq[seq] = phase_by
        self.valid_t_ends_by_seq[seq] = valid_t_ends
        self.transition_t_ends_by_seq[seq] = transition_t_ends
        self.non_transition_t_ends_by_seq[seq] = non_transition_t_ends

        # Build windows (overlapping, deterministic)
        for t_end in valid_t_ends:
            row = {
                'embryo_id': eid,
                'plane': plane,
                't_end': t_end,
                'tgt_frame': t_end + 1,
                'max_frame': max_frame,
            }
            self.windows.append(row)
            self.windows_by_plane.setdefault(plane, []).append(row)
            if t_end in transition_t_ends:
                self.transition_windows_by_plane.setdefault(plane, []).
↪append(row)
            else:
                self.non_transition_windows_by_plane.setdefault(plane, []).
↪append(row)

        if VERBOSE:
            frames_per_seq = [len(self.frames_by_seq[s]) for s in self.
↪sequences]
            valid_tgts_per_seq = [len(self.valid_t_ends_by_seq[s]) for s in
↪self.sequences]
            trans_tgts_per_seq = [len(self.transition_t_ends_by_seq[s]) for s
↪in self.sequences]
            total_frames = int(np.sum(frames_per_seq)) if frames_per_seq else 0
            def _avg(vals):

```

```

        return float(np.mean(vals)) if vals else 0.0
    def _minmax(vals):
        return (int(np.min(vals)), int(np.max(vals))) if vals else (0,
↪0)

    fp_min, fp_max = _minmax(frames_per_seq)
    vt_min, vt_max = _minmax(valid_tgts_per_seq)
    tr_min, tr_max = _minmax(trans_tgts_per_seq)
    print(
        f"[PairDataset:{self.split_name}] sequences={len(self.
↪sequences)} | "
        f"frames={total_frames} | frames/seq avg={_avg(frames_per_seq):.
↪1f} (min={fp_min}, max={fp_max}) | "
        f"valid windows/seq avg={_avg(valid_tgts_per_seq):.1f}
↪(min={vt_min}, max={vt_max}) | "
        f"transition windows/seq avg={_avg(trans_tgts_per_seq):.1f}
↪(min={tr_min}, max={tr_max}) | "
        f"skipped short={self._skipped_short}, skipped no-target={self.
↪_skipped_no_tgt}"
    )

    def __len__(self):
        return len(self.windows)

    def _build_row(self, row: dict):
        seq = (row['embryo_id'], row['plane'])
        t_end = row['t_end']
        ctx_frames = [t_end - i for i in range(CONTEXT_K - 1, -1, -1)]
        ctx_paths = [self.path_by_seq[seq][f] for f in ctx_frames]
        tgt_path = self.path_by_seq[seq][t_end + 1]
        out = dict(row)
        out['ctx_paths'] = ctx_paths
        out['tgt_path'] = tgt_path
        return out

    def sample_pairs_for_plane(self, plane: str, n: int, mode: str = 'any'):
        if mode == 'transition':
            rows = self.transition_windows_by_plane.get(plane, [])
        elif mode == 'non_transition':
            rows = self.non_transition_windows_by_plane.get(plane, [])
        else:
            rows = self.windows_by_plane.get(plane, [])
        if not rows or n <= 0:
            return []
        picks = [rows[self.rng.randint(len(rows))] for _ in range(n)]
        return [self._build_row(r) for r in picks]

```

```

def sample_transition_pairs_for_plane(self, plane: str, n: int):
    return self.sample_pairs_for_plane(plane, n, mode='transition')

def sample_non_transition_pairs_for_plane(self, plane: str, n: int):
    return self.sample_pairs_for_plane(plane, n, mode='non_transition')

def _apply_context_dropout(self, x_ctx: torch.Tensor) -> torch.Tensor:
    """Randomly zero out the oldest 1-3 context frames; never drop t-1.
    Probabilities: full (50%), drop1 (25%), drop2 (15%), drop3 (10%).
    """
    r = self.rng.rand()
    if r < CTX_DROP_P_FULL:
        return x_ctx
    if r < CTX_DROP_P_FULL + CTX_DROP_P_1:
        drop_n = 1
    elif r < CTX_DROP_P_FULL + CTX_DROP_P_1 + CTX_DROP_P_2:
        drop_n = 2
    else:
        drop_n = 3
    if drop_n > 0:
        x_ctx = x_ctx.clone()
        x_ctx[:drop_n] = 0.0 # zero oldest frames only (t-1 is last
↪channel)
    return x_ctx

def __getitem__(self, idx):
    row = self._build_row(self.windows[idx])
    ctx_pils = [load_pil(p) for p in row['ctx_paths']]
    tgt_pil = load_pil(row['tgt_path'])
    ctx_pils, tgt_pil = _augment_stack(ctx_pils, tgt_pil)
    x_ctx = torch.stack([pil_to_tensor(p).squeeze(0) for p in ctx_pils],
↪dim=0) # [k, H, W]
    x_tgt = pil_to_tensor(tgt_pil) # [1, H, W]
    # Context dropout during training only (val/test keep full context)
    if self.split_name == 'train':
        x_ctx = self._apply_context_dropout(x_ctx)
    # Absolute frame index embedding (no normalization)
    if USE_TIME_MAP:
        frame_idx = min(int(row['t_end']), int(MAX_FRAME_INDEX))
        frame_idx = torch.tensor(frame_idx, dtype=torch.long)
        return x_ctx, x_tgt, frame_idx
    return x_ctx, x_tgt

```

```

[9]: # Build datasets/loaders once; pairs are sampled on the fly
train_dataset = PairDataset(index_for_pairs, train_ids, SPLIT_SEED, 'train')
val_dataset = PairDataset(index_for_pairs, val_ids, SPLIT_SEED, 'val')
test_dataset = PairDataset(index_for_pairs, test_ids, SPLIT_SEED, 'test')

```

```

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True,
    ↪num_workers=NUM_WORKERS, pin_memory=True, persistent_workers=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False,
    ↪num_workers=NUM_WORKERS, pin_memory=True)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False,
    ↪num_workers=NUM_WORKERS, pin_memory=True)

```

```

[PairDataset:train] sequences=7 | frames=810 | frames/seq avg=115.7 (min=114,
max=116) | valid windows/seq avg=99.7 (min=86, max=102) | transition windows/seq
avg=5.7 (min=4, max=6) | skipped short=0, skipped no-target=0
[PairDataset:val] sequences=7 | frames=3108 | frames/seq avg=444.0 (min=444,
max=444) | valid windows/seq avg=437.0 (min=437, max=437) | transition
windows/seq avg=10.0 (min=10, max=10) | skipped short=0, skipped no-target=0
[PairDataset:test] sequences=7 | frames=1469 | frames/seq avg=209.9 (min=209,
max=210) | valid windows/seq avg=201.9 (min=195, max=203) | transition
windows/seq avg=8.0 (min=8, max=8) | skipped short=0, skipped no-target=0

```

```

[10]: diffusion = DiffusionCond(context_k=CONTEXT_K, use_time_map=USE_TIME_MAP).
    ↪to(device)
if USE_GRAD_CHECKPOINTING:
    diffusion.unet.enable_gradient_checkpointing()
optimizer = torch.optim.AdamW(diffusion.parameters(), lr=LR, betas=(0.9, 0.999))

# Avoid the noisiest tail for classifier guidance (align with classifier
    ↪training)
GUIDANCE_T_MAX = max(1, int(0.9 * TIMESTEPS))

def _repeat_channels(x, target_channels=3):
    """Repeat single-channel tensors along channel dim."""
    if x.shape[1] == target_channels:
        return x
    return x.repeat(1, target_channels, 1, 1)

def _resize_for_classifier(x):
    """Resize [-1,1] tensors to the classifier input size."""
    if x.shape[-1] != CLASSIFIER_INPUT_SIZE:
        return F.interpolate(x, size=(CLASSIFIER_INPUT_SIZE,
    ↪CLASSIFIER_INPUT_SIZE), mode='bilinear', align_corners=False)
    return x

def _soft_targets_from_indices(indices: torch.Tensor, num_classes: int, sigma:
    ↪float):
    """Gaussian soft targets over ordered classes."""
    idx = torch.arange(num_classes, device=indices.device, dtype=torch.float32)
    centers = indices.float().unsqueeze(1)
    dist2 = (idx.unsqueeze(0) - centers) ** 2

```

```

weights = torch.exp(-dist2 / (2 * (sigma ** 2)))
weights = weights / weights.sum(dim=1, keepdim=True)
return weights

def _classifier_guidance_step(
    x: torch.Tensor,
    cls: torch.Tensor,
    t_tensor: torch.Tensor,
    classifier: nn.Module,
    classifier_scale: float,
):
    """One external classifier guidance step on current x (shape [B,1,H,W])."""
    B = x.size(0)
    with torch.enable_grad():
        x_noisy_part = x.detach().requires_grad_(True)
        if getattr(classifier, 'handles_resize', False):
            x_for_clf = x_noisy_part
        else:
            x_for_clf = _resize_for_classifier(x_noisy_part)
        if not getattr(classifier, 'accepts_normalized', False):
            x_for_clf = (x_for_clf + 1) / 2
            if x_for_clf.shape[1] == 1:
                x_for_clf = _repeat_channels(x_for_clf, 3)
        if getattr(classifier, 'accepts_timestep', False):
            logits = classifier(x_for_clf, t_tensor)
        else:
            logits = classifier(x_for_clf)
        logp = F.log_softmax(logits, dim=1)
        target_indices = cls.view(-1)
        if GUIDANCE_USE_SOFT_TARGETS and GUIDANCE_SMOOTH_SIGMA > 0:
            soft = _soft_targets_from_indices(target_indices, NUM_PHASES,
↳GUIDANCE_SMOOTH_SIGMA)
            loss = -(soft * logp).sum(dim=1).mean()
        else:
            loss = -logp[torch.arange(B, device=device), target_indices].mean()
            grad = torch.autograd.grad(loss, x_noisy_part, retain_graph=False,
↳create_graph=False)[0]
            updated = (x_noisy_part - classifier_scale * grad).clamp(-1.0, 1.0)
    return updated.detach()

def _eps_from_model(x_noisy: torch.Tensor, x_ctx: torch.Tensor, t_tensor: torch.
↳Tensor, time_map: torch.Tensor | None = None):
    x_in = diffusion.build_input(x_noisy, x_ctx, time_map=time_map)
    return diffusion.forward(x_in, t_tensor)

def _ensure_context_batch(x_ctx: torch.Tensor | None, shape):
    """Ensure context tensor is [B, k, H, W]."""

```

```

B, H, W = shape[0], shape[2], shape[3]
if x_ctx is None:
    return torch.zeros((B, CONTEXT_K, H, W), device=device)
x_ctx = x_ctx.to(device)
if x_ctx.dim() == 3:
    x_ctx = x_ctx.unsqueeze(0)
if x_ctx.shape[1] != CONTEXT_K:
    raise ValueError(f"Expected context with k={CONTEXT_K} channels, got {x_ctx.shape[1]}")
return x_ctx

def _ensure_time_map(frame_idx: torch.Tensor | int | None, shape):
    if not USE_TIME_MAP:
        return None
    if frame_idx is None:
        raise ValueError("frame_idx is required when USE_TIME_MAP=True")
    if not torch.is_tensor(frame_idx):
        frame_idx = torch.tensor(frame_idx, device=device, dtype=torch.long)
    if frame_idx.dim() == 0:
        frame_idx = frame_idx.view(1)
    frame_idx = frame_idx.clamp(0, MAX_FRAME_INDEX).long()
    return diffusion.build_time_map(frame_idx, shape[2], shape[3])

def ddim_sample_from_model(
    shape, x_ctx, frame_idx=None,
    classifier=None, classifier_scale=CLASSIFIER_GUIDANCE_SCALE,
    steps=DDIM_STEPS, eta=DDIM_ETA,
    cls: torch.Tensor | None = None
):
    """DDIM sampling with optional classifier guidance (external only)."""
    B = shape[0]
    x = torch.randn((B, 1, shape[2], shape[3]), device=device)
    x_ctx = _ensure_context_batch(x_ctx, shape)
    time_map = _ensure_time_map(frame_idx, shape)
    ddim_scheduler.set_timesteps(steps, device=device)
    for t in ddim_scheduler.timesteps:
        t_tensor = torch.full((B,), int(t), device=device, dtype=torch.long)
        with torch.no_grad():
            eps = _eps_from_model(x, x_ctx, t_tensor, time_map=time_map)
            x = ddim_scheduler.step(eps, t, x, eta=eta).prev_sample
            if classifier is not None and cls is not None and int(t) %
GUIDANCE_STEP_EVERY == 0 and 0 < int(t) < GUIDANCE_T_MAX:
                x = _classifier_guidance_step(x, cls, t_tensor, classifier,
classifier_scale)
    return x.detach()

def ddim_invert(x0, x_ctx, frame_idx=None, steps=INVERSION_STEPS):

```

```

"""DDIM inversion: map  $x_0 \rightarrow x_T$  deterministically ( $\eta=0$ )."""
B = x0.size(0)
x_ctx = _ensure_context_batch(x_ctx, (B, 1, x0.shape[-2], x0.shape[-1]))
time_map = _ensure_time_map(frame_idx, (B, 1, x0.shape[-2], x0.shape[-1]))
x = x0
ddim_scheduler.set_timesteps(steps, device=device)
timesteps = list(reversed(ddim_scheduler.timesteps))
alphas_cumprod = ddim_scheduler.alphas_cumprod.to(device)
for i, t in enumerate(timesteps[:-1]):
    t_next = timesteps[i + 1]
    t_tensor = torch.full((B,), int(t), device=device, dtype=torch.long)
    with torch.no_grad():
        eps = _eps_from_model(x, x_ctx, t_tensor, time_map=time_map)
        alpha_t = alphas_cumprod[int(t)].view(1, 1, 1, 1)
        alpha_next = alphas_cumprod[int(t_next)].view(1, 1, 1, 1)
        sqrt_alpha_t = torch.sqrt(alpha_t)
        sqrt_one_minus_alpha_t = torch.sqrt(1.0 - alpha_t)
        x0_pred = (x - sqrt_one_minus_alpha_t * eps) / sqrt_alpha_t
        x = torch.sqrt(alpha_next) * x0_pred + torch.sqrt(1.0 - alpha_next) * ␣
↪eps
    return x.detach()

def ddim_reconstruct_from_xT(
    x_T, x_ctx, frame_idx=None,
    classifier=None, classifier_scale=CLASSIFIER_GUIDANCE_SCALE, ␣
↪steps=INVERSION_STEPS,
    cls: torch.Tensor | None = None
):
    """DDIM denoising from  $x_T$  back to  $x_0$  (supports gradients)."""
    B = x_T.size(0)
    x_ctx = _ensure_context_batch(x_ctx, (B, 1, x_T.shape[-2], x_T.shape[-1]))
    time_map = _ensure_time_map(frame_idx, (B, 1, x_T.shape[-2], x_T.shape[-1]))
    x = x_T
    ddim_scheduler.set_timesteps(steps, device=device)
    for t in ddim_scheduler.timesteps:
        t_tensor = torch.full((B,), int(t), device=device, dtype=torch.long)
        eps = _eps_from_model(x, x_ctx, t_tensor, time_map=time_map)
        x = ddim_scheduler.step(eps, t, x,  $\eta=0.0$ ).prev_sample
        if classifier is not None and cls is not None and int(t) % ␣
↪GUIDANCE_STEP_EVERY == 0 and 0 < int(t) < GUIDANCE_T_MAX:
            x = _classifier_guidance_step(x, cls, t_tensor, classifier, ␣
↪classifier_scale)
    return x

def sample_from_model(shape, x_ctx, frame_idx=None,
    classifier=None, ␣
↪classifier_scale=CLASSIFIER_GUIDANCE_SCALE, cls: torch.Tensor | None = None):

```

```

        """Generate samples using DDIM (optionally with external classifier_
↳guidance)."""
        return ddim_sample_from_model(
            shape, x_ctx, frame_idx=frame_idx,
            classifier=classifier, classifier_scale=classifier_scale,
↳steps=DDIM_STEPS, eta=DDIM_ETA, cls=cls
        )

```

```

[11]: # Try to auto-detect a phase classifier checkpoint
from torchvision.models import efficientnet_v2_s

class SinusoidalTimeEmbedding(nn.Module):
    def __init__(self, dim: int, max_period: float = 10000.0):
        super().__init__()
        self.dim = dim
        self.max_period = max_period
    def forward(self, t: torch.Tensor):
        # t: [B] int64 timesteps
        half = self.dim // 2
        t = t.float() / max(1, Timesteps - 1) # normalize to [0,1]
        freqs = torch.exp(
            -math.log(self.max_period) * torch.arange(0, half, device=t.device).
↳float() / half
        )
        args = t[:, None] * freqs[None] * 2 * math.pi
        emb = torch.cat([torch.sin(args), torch.cos(args)], dim=1)
        if self.dim % 2 == 1:
            emb = torch.cat([emb, torch.zeros_like(emb[:, :1])], dim=1)
        return emb

class TimeCondEfficientNet(nn.Module):
    def __init__(self, n_classes: int, time_dim: int = 128):
        super().__init__()
        self.base = efficientnet_v2_s(weights=None)
        # Convert first layer to grayscale while preserving pre-trained weights
        old_conv = self.base.features[0][0]
        new_conv = nn.Conv2d(1, old_conv.out_channels, kernel_size=3, stride=2,
↳padding=1, bias=False)
        with torch.no_grad():
            new_conv.weight.data = old_conv.weight.data.sum(dim=1,
↳keepdim=True) / 3.0
        self.base.features[0][0] = new_conv

        in_feats = self.base.classifier[1].in_features
        self.base.classifier = nn.Identity()

        self.time_embed = SinusoidalTimeEmbedding(time_dim)

```

```

        self.time_mlp = nn.Sequential(
            nn.Linear(time_dim, time_dim),
            nn.SiLU(),
            nn.Linear(time_dim, time_dim),
        )

        self.classifier = nn.Sequential(
            nn.Dropout(p=0.2),
            nn.Linear(in_feats + time_dim, n_classes),
        )

    def forward(self, x: torch.Tensor, t: torch.Tensor):
        feats = self.base.features(x)
        feats = self.base.avgpool(feats)
        feats = torch.flatten(feats, 1)
        t_emb = self.time_mlp(self.time_embed(t))
        feats = torch.cat([feats, t_emb], dim=1)
        return self.classifier(feats)

def autodetect_model(model_dir: Path, keywords=('phase', 'classifier', 'model', '.
    ↪pt', '.pth', '.ckpt', '.jit')):
    if not model_dir.exists():
        return None
    candids = []
    for p in model_dir.glob('**/*'):
        name = p.name.lower()
        if p.is_file() and any(k in name for k in keywords):
            candids.append(p)
    if candids:
        candids.sort(key=lambda p: p.stat().st_mtime, reverse=True)
        return candids[0]
    return None

if PHASE_MODEL_PATH is None:
    PHASE_MODEL_PATH = autodetect_model(PHASE_MODEL_DIR)
print('Phase classifier candidate:', PHASE_MODEL_PATH)

# We don't know your exact classifier class; we attempt a few robust load paths.
phase_classifier = None
load_errors: list[str] = []

if PHASE_MODEL_PATH is not None and USE_CLASSIFIER_GUIDANCE:
    add_safe_globals([EfficientNet, TimeCondEfficientNet])
    try:
        phase_classifier = torch.jit.load(str(PHASE_MODEL_PATH), ↵
    ↪map_location=device)
        phase_classifier.eval()

```

```

except Exception as exc:
    load_errors.append(f"TorchScript load error: {exc}")
    try:
        obj = torch.load(PHASE_MODEL_PATH, map_location=device,
↪weights_only=False)
        if isinstance(obj, nn.Module):
            phase_classifier = obj
        elif isinstance(obj, dict):
            if isinstance(obj.get('model'), nn.Module):
                phase_classifier = obj['model']
            elif isinstance(obj.get('state_dict'), dict):
                load_errors.append("Checkpoint contains a state_dict;
↪please instantiate the architecture before loading.")
                phase_classifier = None
            else:
                load_errors.append(f"Unsupported checkpoint payload keys:
↪{list(obj.keys())}")
                phase_classifier = None
            else:
                load_errors.append(f"Unsupported checkpoint payload type:
↪{type(obj)}")
                phase_classifier = None
            if isinstance(phase_classifier, nn.Module):
                phase_classifier.eval()
        except Exception as exc2:
            load_errors.append(f"torch.load error: {exc2}")
            phase_classifier = None

if isinstance(phase_classifier, nn.Module):
    class PhaseClassifierWrapper(nn.Module):
        def __init__(self, model: nn.Module):
            super().__init__()
            self.model = model
            self.accepts_normalized = True
            self.accepts_timestep = True
            self.handles_resize = True
        def forward(self, x, t=None):
            # x: [B,1,H,W] in [-1,1] (from diffusion model)
            x = _resize_for_classifier(x)
            if getattr(self.model, 'in_channels', 1) == 3 or any(getattr(m,
↪'in_channels', 1) == 3 for m in self.model.modules() if hasattr(m,
↪'in_channels'))):
                x = _repeat_channels(x, 3)
            if t is None:
                t = torch.zeros(x.size(0), device=x.device, dtype=torch.long)
            return self.model(x, t)
    phase_classifier = PhaseClassifierWrapper(phase_classifier).to(device)

```

```

    phase_classifier.eval()
    print('Phase classifier loaded for external guidance.')
else:
    if load_errors:
        for err in load_errors:
            print(' ->', err)
        print('Phase classifier not available or failed to load. External guidance_
↳will be disabled.')

def save_tensor_image(tensor, path):
    """Save tensor as grayscale image."""
    arr = (denorm(tensor).clamp(0, 1).cpu().numpy() * 255).astype(np.uint8)
    if arr.ndim == 3 and arr.shape[0] == 1:
        arr = arr[0]
    Image.fromarray(arr).save(path)

lpips_metric = None
if COMPUTE_VAL_METRICS:
    lpips_metric = LPIPS(net='alex').to(device)
    lpips_metric.eval()
    for p in lpips_metric.parameters():
        p.requires_grad_(False)

_gaussian_kernel_cache: dict[tuple[int, float, str, int | None, torch.dtype],
↳torch.Tensor] = {}
def _gaussian_kernel(window_size: int, sigma: float, device: torch.device,
↳dtype: torch.dtype):
    key = (window_size, sigma, device.type, device.index, dtype)
    kernel = _gaussian_kernel_cache.get(key)
    if kernel is None:
        coords = torch.arange(window_size, device=device, dtype=dtype) -
↳window_size // 2
        gauss = torch.exp(-(coords ** 2) / (2 * sigma ** 2))
        gauss /= gauss.sum()
        kernel_1d = gauss.unsqueeze(0)
        kernel_2d = kernel_1d.t() @ kernel_1d
        kernel_2d /= kernel_2d.sum()
        kernel = kernel_2d.unsqueeze(0).unsqueeze(0)
        _gaussian_kernel_cache[key] = kernel
    return kernel

def compute_ssim_metric(pred: torch.Tensor, target: torch.Tensor, window_size:
↳int = 11, sigma: float = 1.5) -> torch.Tensor:
    pred = pred.clamp(-1, 1)
    target = target.clamp(-1, 1)
    pred_01 = (pred + 1) / 2
    target_01 = (target + 1) / 2

```

```

device = pred.device
dtype = pred.dtype
channels = pred_01.size(1)
kernel = _gaussian_kernel(window_size, sigma, device, dtype).
↳expand(channels, 1, window_size, window_size)
padding = window_size // 2
mu_pred = F.conv2d(pred_01, kernel, padding=padding, groups=channels)
mu_target = F.conv2d(target_01, kernel, padding=padding, groups=channels)
mu_pred_sq = mu_pred.pow(2)
mu_target_sq = mu_target.pow(2)
mu_pred_target = mu_pred * mu_target
sigma_pred = F.conv2d(pred_01 * pred_01, kernel, padding=padding,↳
↳groups=channels) - mu_pred_sq
sigma_target = F.conv2d(target_01 * target_01, kernel, padding=padding,↳
↳groups=channels) - mu_target_sq
sigma_pred_target = F.conv2d(pred_01 * target_01, kernel, padding=padding,↳
↳groups=channels) - mu_pred_target
sigma_pred = torch.clamp_min(sigma_pred, 0.0)
sigma_target = torch.clamp_min(sigma_target, 0.0)
c1 = (0.01) ** 2
c2 = (0.03) ** 2
numerator = (2 * mu_pred_target + c1) * (2 * sigma_pred_target + c2)
denominator = (mu_pred_sq + mu_target_sq + c1) * (sigma_pred + sigma_target,↳
↳+ c2)
ssim_map = numerator / (denominator + 1e-6)
return ssim_map.mean()

def compute_lpips_metric(pred: torch.Tensor, target: torch.Tensor) -> torch.
↳Tensor:
    if lpips_metric is None:
        raise RuntimeError('LPIPS metric requested but the LPIPS model is not,↳
↳initialised.')
    pred_in = _repeat_channels(pred.clamp(-1, 1))
    target_in = _repeat_channels(target.clamp(-1, 1))
    with torch.no_grad():
        return lpips_metric(pred_in, target_in, normalize=False).mean()

```

Phase classifier candidate: /mnt/c/Users/ioann/OneDrive/Documents/Projects/thesis/classifier/embryo_phase_models/phase_classifier_all_focal_planes_f1=600.pth
Phase classifier not available or failed to load. External guidance will be disabled.

Setting up [LPIPS] perceptual loss: trunk [alex], v[0.1], spatial [off]

/home/khasion/miniconda3/envs/thesis/lib/python3.10/site-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.
warnings.warn(

```

/home/khasion/miniconda3/envs/thesis/lib/python3.10/site-
packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a
weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed
in the future. The current behavior is equivalent to passing
`weights=AlexNet_Weights.IMAGENET1K_V1`. You can also use
`weights=AlexNet_Weights.DEFAULT` to get the most up-to-date weights.
  warnings.warn(msg)

```

```

Loading model from: /home/khasion/miniconda3/envs/thesis/lib/python3.10/site-
packages/lpips/weights/v0.1/alex.pth

```

```

[12]: # Train loop - context-conditioned diffusion model
best_val = None
ckpt_path = OUT_MODELS_DIR / f'nextframe_ctxk{CONTEXT_K}_{IMG_SIZE}px.pt'

# Early stopping config
EARLY_STOPPING_PATIENCE = 10 # set to None or 0 to disable
epochs_no_improve = 0

# GradScaler for proper mixed precision training (prevents NaN from underflow/
↳ overflow)
use_amp = torch.cuda.is_available()
scaler = torch.amp.GradScaler('cuda', enabled=use_amp)
metric_history = []

def one_epoch(loader, train=True, desc: str | None = None):
    diffusion.train(mode=train)
    total = 0.0
    count = 0
    nan_batches = 0
    inv_total = 0.0
    inv_count = 0
    if desc is None:
        desc = "Train" if train else "Val"
    for batch_idx, batch in enumerate(tqdm_bar(loader, desc=desc)):
        if USE_TIME_MAP:
            x_ctx, x_tgt, frame_idx = batch
            frame_idx = frame_idx.to(device, non_blocking=True)
        else:
            x_ctx, x_tgt = batch
            frame_idx = None
        x_ctx = x_ctx.to(device, non_blocking=True) # [B, k, H, W]
        x_tgt = x_tgt.to(device, non_blocking=True) # [B, 1, H, W]
        time_map = diffusion.build_time_map(frame_idx, x_ctx.shape[-2], x_ctx.
↳ shape[-1]) if USE_TIME_MAP else None

        # Mixed precision training with proper GradScaler
        with torch.autocast(device_type='cuda', enabled=use_amp):

```

```

        mse_loss, x0_pred = diffusion.p_losses(x_tgt, x_ctx,
↪time_map=time_map)
        loss = mse_loss
        if lpips_metric is not None:
            lpips_in = _repeat_channels(x0_pred.clamp(-1, 1))
            tgt_in = _repeat_channels(x_tgt.clamp(-1, 1))
            lpips_loss = lpips_metric(lpips_in, tgt_in, normalize=False).
↪mean()

            loss = loss + LPIPS_LOSS_WEIGHT * lpips_loss

        # Optional inversion training (DDIM inversion + reconstruction)
        inv_loss = None
        if INVERSION_TRAINING and train and (batch_idx % max(1,
↪INVERSION_EVERY_N) == 0):
            b_inv = min(x_tgt.size(0), INVERSION_MAX_BATCH)
            x_ctx_inv = x_ctx[:b_inv]
            x_tgt_inv = x_tgt[:b_inv]
            frame_idx_inv = frame_idx[:b_inv] if frame_idx is not None else None
            with torch.no_grad():
                x_T = ddim_invert(
                    x_tgt_inv, x_ctx_inv, frame_idx=frame_idx_inv,
                    steps=INVERSION_STEPS
                )
            with torch.autocast(device_type='cuda', enabled=use_amp):
                x_recon = ddim_reconstruct_from_xT(
                    x_T, x_ctx_inv, frame_idx=frame_idx_inv,
                    classifier=None, steps=INVERSION_STEPS, cls=None
                )
            inv_loss = F.l1_loss(x_recon, x_tgt_inv)
            loss = loss + INVERSION_LAMBDA * inv_loss

        # Check for NaN/Inf BEFORE backward (critical: don't corrupt model with
↪bad gradients)
        if torch.isnan(loss) or torch.isinf(loss):
            nan_batches += 1
            if nan_batches <= 3: # Only warn first few times
                print(f"WARNING: NaN/Inf loss detected, skipping batch (total:
↪{nan_batches})")
            continue

        if train:
            optimizer.zero_grad(set_to_none=True)
            scaler.scale(loss).backward()
            scaler.unscale_(optimizer)
            torch.nn.utils.clip_grad_norm_(diffusion.parameters(), GRAD_CLIP)
            scaler.step(optimizer)
            scaler.update()

```

```

        total += loss.item() * x_tgt.size(0)
        count += x_tgt.size(0)
        if inv_loss is not None:
            inv_total += inv_loss.item() * x_tgt.size(0)
            inv_count += x_tgt.size(0)

    if nan_batches > 0:
        print(f" (Epoch had {nan_batches} NaN/Inf batches skipped)")
    avg = total / max(1, count)
    inv_avg = inv_total / max(1, inv_count) if inv_count > 0 else None
    return avg, inv_avg

def _rows_to_tensors(rows: list[dict]):
    ctx_list = []
    tgt_list = []
    frame_idx_list = []
    for row in rows:
        ctx_tensors = [load_image(p).squeeze(0) for p in row['ctx_paths']] # [k, H, W]
        x_ctx = torch.stack(ctx_tensors, dim=0)
        x_tgt = load_image(row['tgt_path']) # [1, H, W]
        ctx_list.append(x_ctx)
        tgt_list.append(x_tgt)
        if USE_TIME_MAP:
            frame_idx = min(int(row['t_end']), int(MAX_FRAME_INDEX))
            frame_idx_list.append(frame_idx)
    x_ctx = torch.stack(ctx_list, dim=0)
    x_tgt = torch.stack(tgt_list, dim=0)
    if USE_TIME_MAP:
        frame_idx = torch.tensor(frame_idx_list, dtype=torch.long)
        return x_ctx, x_tgt, frame_idx
    return x_ctx, x_tgt

def _compute_metrics_from_rows(rows: list[dict], classifier_for_metrics=None, desc: str = "Val metrics"):
    if not rows:
        return None, None
    ssim_vals = []
    lpips_vals = []
    for i in tqdm_bar(range(0, len(rows), BATCH_SIZE), desc=desc):
        batch_rows = rows[i:i + BATCH_SIZE]
        if USE_TIME_MAP:
            x_ctx, x_tgt, frame_idx = _rows_to_tensors(batch_rows)
            frame_idx = frame_idx.to(device, non_blocking=True)
        else:
            x_ctx, x_tgt = _rows_to_tensors(batch_rows)

```

```

        frame_idx = None
    x_ctx = x_ctx.to(device, non_blocking=True)
    x_tgt = x_tgt.to(device, non_blocking=True)
    x_gen = sample_from_model(
        (x_ctx.size(0), 1, IMG_SIZE, IMG_SIZE),
        x_ctx=x_ctx,
        frame_idx=frame_idx,
        classifier=classifier_for_metrics,
        cls=None,
    )
    ssim_vals.append(compute_ssim_metric(x_gen, x_tgt).item())
    if lpips_metric is not None:
        lpips_vals.append(compute_lpips_metric(x_gen, x_tgt).item())
    ssim = float(np.mean(ssim_vals)) if ssim_vals else None
    lpips = float(np.mean(lpips_vals)) if lpips_vals else None
    return ssim, lpips

def _collect_pairs_for_metrics(dataset: PairDataset, mode: str, max_samples:
    int):
    rows = []
    planes = list(dataset.windows_by_plane.keys())
    for plane in tqdm_bar(planes, desc=f"Collecting {mode} pairs"):
        if len(rows) >= max_samples:
            break
        n_needed = max_samples - len(rows)
        if mode == 'transition':
            rows.extend(dataset.sample_transition_pairs_for_plane(plane,
                n_needed))
        elif mode == 'non_transition':
            rows.extend(dataset.sample_non_transition_pairs_for_plane(plane,
                n_needed))
        else:
            rows.extend(dataset.sample_pairs_for_plane(plane, n_needed))
    return rows

def _save_sample_grid(rows: list[dict], out_dir: Path, title_suffix: str,
    classifier_for_infer=None):
    if not rows:
        return
    out_dir.mkdir(parents=True, exist_ok=True)
    for idx, row in enumerate(tqdm_bar(rows, desc=f"Saving samples
        {title_suffix}")):
        ctx_tensors = [load_image(p).squeeze(0) for p in row['ctx_paths']]
        tgt_tensor_cpu = load_image(row['tgt_path'])
        x_ctx = torch.stack(ctx_tensors, dim=0).unsqueeze(0).to(device)
        frame_idx = None
        if USE_TIME_MAP:

```

```

        frame_idx = min(int(row['t_end']), int(MAX_FRAME_INDEX))
        frame_idx = torch.tensor([frame_idx], device=device, dtype=torch.
↳long)
    x_gen = sample_from_model(
        (1, 1, IMG_SIZE, IMG_SIZE),
        x_ctx=x_ctx,
        frame_idx=frame_idx,
        classifier=classifier_for_infer,
        cls=None,
    )
    # number of panels: k context + target + generated
    ncols = CONTEXT_K + 2
    fig, axes = plt.subplots(1, ncols, figsize=(3 * ncols, 3))
    if ncols == 1:
        axes = [axes]

    visuals = []
    titles = []

    # Context frames: ctx_tensors[i] are [H,W] in [-1,1]
    for i, ctx in enumerate(ctx_tensors):
        # i=0 corresponds to t-k, i=k-1 corresponds to t-1
        img = denorm(ctx.unsqueeze(0).unsqueeze(0))[0, 0].cpu().numpy()
        visuals.append(img)
        titles.append(f"Ctx (t- $\{CONTEXT\_K - i\}$ )")

    # Target frame: tgt_tensor_cpu is [1,H,W] in [-1,1]
    tgt_img = denorm(tgt_tensor_cpu.unsqueeze(0))[0, 0].cpu().numpy()
    visuals.append(tgt_img)
    titles.append("Target (t+1)")

    # Generated frame: x_gen is [1,1,H,W] in [-1,1]
    gen_img = denorm(x_gen)[0, 0].detach().cpu().numpy()
    visuals.append(gen_img)
    titles.append("Generated")

    for ax, img, title in zip(axes, visuals, titles):
        ax.imshow(img, cmap="gray")
        ax.set_title(title)
        ax.axis("off")

    fig.suptitle(f"{row['embryo_id']} ({row['plane']}) - context_
↳k= $\{CONTEXT\_K\}$  → next {title_suffix}", fontsize=10)
    fig.tight_layout()
    save_path = out_dir / f'sample_{idx:02d}.png'
    fig.savefig(save_path, dpi=150)
    plt.close(fig)

```

```

def _save_rollout_grid(row: dict, out_dir: Path, steps: int,
    classifier_for_infer=None):
    out_dir.mkdir(parents=True, exist_ok=True)
    ctx_tensors = [load_image(p).squeeze(0) for p in row['ctx_paths']]
    x_ctx = torch.stack(ctx_tensors, dim=0).unsqueeze(0).to(device)
    t_end = int(row['t_end'])

    generated = []
    frame_idx = min(int(t_end), int(MAX_FRAME_INDEX))
    for s in tqdm_bar(range(steps), desc=f"Rollout sampling ({row['embryo_id']})
    {row['plane']})"):
        if USE_TIME_MAP:
            frame_idx = min(frame_idx, int(MAX_FRAME_INDEX))
            frame_idx_tensor = torch.tensor([frame_idx], device=device,
dtype=torch.long)
        else:
            frame_idx_tensor = None
        x_gen = sample_from_model(
            (1, 1, IMG_SIZE, IMG_SIZE),
            x_ctx=x_ctx,
            frame_idx=frame_idx_tensor,
            classifier=classifier_for_infer,
            cls=None,
        )
        generated.append(x_gen[0].detach().cpu())
        # update context window: drop oldest, append generated
        x_ctx = torch.cat([x_ctx[:, 1:], x_gen.detach()], dim=1)
        frame_idx += 1

    ncols = CONTEXT_K + steps
    fig, axes = plt.subplots(1, ncols, figsize=(3 * ncols, 3))
    if ncols == 1:
        axes = [axes]

    # Context frames
    for i, ctx in enumerate(ctx_tensors):
        img = denorm(ctx.unsqueeze(0))[0].cpu().numpy()
        axes[i].imshow(img, cmap="gray")
        axes[i].set_title(f"Ctx (t-{CONTEXT_K - i})")
        axes[i].axis("off")

    # Generated rollout frames
    for i, gen in enumerate(generated):
        img = denorm(gen)[0].cpu().numpy()
        axes[CONTEXT_K + i].imshow(img, cmap="gray")
        axes[CONTEXT_K + i].set_title(f"Gen (t+{i+1})")

```

```

        axes[CONTEXT_K + i].axis("off")

    fig.suptitle(f"{row['embryo_id']} ({row['plane']}) - rollout {steps}
↳steps", fontsize=10)
    fig.tight_layout()
    save_path = out_dir / f"rollout_t{row['t_end']:05d}_k{CONTEXT_K}_h{steps}.
↳png"
    fig.savefig(save_path, dpi=150)
    plt.close(fig)

for epoch in range(1, EPOCHS+1):
    tl, tl_inv = one_epoch(train_loader, train=True, desc=f"Epoch {epoch:03d}
↳Train")
    with torch.no_grad():
        vl, _ = one_epoch(val_loader, train=False, desc=f"Epoch {epoch:03d}
↳Val")

    val_ssim = None
    val_lpips = None
    val_ssim_trans = None
    val_lpips_trans = None
    val_ssim_non = None
    val_lpips_non = None
    if COMPUTE_VAL_METRICS and VAL_METRIC_SAMPLES > 0:
        diffusion.eval()
        classifier_for_metrics = phase_classifier if (USE_CLASSIFIER_GUIDANCE
↳and phase_classifier is not None) else None

        # Overall metrics (mixed transition + non-transition)
        all_rows = _collect_pairs_for_metrics(val_dataset, mode='any',
↳max_samples=VAL_METRIC_SAMPLES)
        val_ssim, val_lpips = _compute_metrics_from_rows(all_rows,
↳classifier_for_metrics, desc="Val metrics (all)")

        # Transitional-only metrics
        trans_rows = _collect_pairs_for_metrics(val_dataset, mode='transition',
↳max_samples=VAL_METRIC_SAMPLES)
        val_ssim_trans, val_lpips_trans =
↳_compute_metrics_from_rows(trans_rows, classifier_for_metrics, desc="Val
↳metrics (transition)")

        # Non-transitional metrics
        non_rows = _collect_pairs_for_metrics(val_dataset,
↳mode='non_transition', max_samples=VAL_METRIC_SAMPLES)
        val_ssim_non, val_lpips_non = _compute_metrics_from_rows(non_rows,
↳classifier_for_metrics, desc="Val metrics (non-transition)")

```

```

log_msg = f"[Epoch {epoch:03d}] train {tl:.4f} | val {vl:.4f}"
if tl_inv is not None:
    log_msg += f" | inv {tl_inv:.4f}"
if val_ssim is not None:
    log_msg += f" | SSIM {val_ssim:.4f}"
if val_ssim_trans is not None:
    log_msg += f" | SSIM_tr {val_ssim_trans:.4f}"
if val_ssim_non is not None:
    log_msg += f" | SSIM_non {val_ssim_non:.4f}"
if val_lpips is not None:
    log_msg += f" | LPIPS {val_lpips:.4f}"
if val_lpips_trans is not None:
    log_msg += f" | LPIPS_tr {val_lpips_trans:.4f}"
if val_lpips_non is not None:
    log_msg += f" | LPIPS_non {val_lpips_non:.4f}"
print(log_msg)
metric_history.append({
    'train_loss': tl,
    'val_loss': vl,
    'train_inv': tl_inv,
    'val_ssim': val_ssim,
    'val_ssim_tr': val_ssim_trans,
    'val_ssim_non': val_ssim_non,
    'val_lpips': val_lpips,
    'val_lpips_tr': val_lpips_trans,
    'val_lpips_non': val_lpips_non
})

if best_val is None or vl < best_val - 1e-8:
    best_val = vl
    epochs_no_improve = 0
    torch.save({'model': diffusion.state_dict(), 'epoch': epoch, 'val_loss':
↪ vl, 'config': {
        'IMG_SIZE': IMG_SIZE, 'Timesteps': Timesteps, 'PHASE_LABELS': ↵
↪ PHASE_LABELS,
        'FOCAL_PLANES': FOCAL_PLANES, 'NUM_PLANES': NUM_PLANES,
        'DDIM_STEPS': DDIM_STEPS, 'DDIM_ETA': DDIM_ETA, 'CONTEXT_K': ↵
↪ CONTEXT_K,
    }}, ckpt_path)
    print('Saved best to', ckpt_path)
else:
    epochs_no_improve += 1
    print(f"No improvement for {epochs_no_improve} epoch(s)")

# Generate samples after every N epochs
print(f"\n--- Generating samples for epoch {epoch} ---")

```

```

diffusion.eval()
classifier_for_infer = phase_classifier if (USE_CLASSIFIER_GUIDANCE and
↳phase_classifier is not None) else None
    with torch.no_grad():
        for plane in tqdm_bar(SAMPLE_PLANES, desc=f"Epoch {epoch:03d} sample
↳outputs"):
            sample_dir = OUT_SAMPLES_DIR / f'epoch_{epoch:03d}' / plane
            sample_dir.mkdir(parents=True, exist_ok=True)
            samples = val_dataset.sample_pairs_for_plane(plane,
↳SAMPLES_PER_PLANE)
            if samples:
                _save_sample_grid(samples, sample_dir, title_suffix='(mixed)',
↳classifier_for_infer=classifier_for_infer)
                # Transitional-only samples
                trans_dir = sample_dir / 'transitions'
                trans_samples = val_dataset.
↳sample_transition_pairs_for_plane(plane, SAMPLES_PER_PLANE)
                if trans_samples:
                    _save_sample_grid(trans_samples, trans_dir,
↳title_suffix='(transition)', classifier_for_infer=classifier_for_infer)
                    # Rollout grids (full descriptive rollout of k steps)
                    rollout_dir = sample_dir / 'rollouts'
                    rollout_seed = val_dataset.sample_pairs_for_plane(plane, 1)
                    if rollout_seed:
                        _save_rollout_grid(rollout_seed[0], rollout_dir,
↳steps=CONTEXT_K, classifier_for_infer=classifier_for_infer)
                    print(f"Samples saved to {OUT_SAMPLES_DIR / f'epoch_{epoch:03d}'}\n")

            # Early stopping check
            if EARLY_STOPPING_PATIENCE and EARLY_STOPPING_PATIENCE > 0 and
↳epochs_no_improve >= EARLY_STOPPING_PATIENCE:
                print(f"Stopping early after {epochs_no_improve} epochs without
↳improvement (patience={EARLY_STOPPING_PATIENCE}).")
                break

print(f"\nTraining complete! Best checkpoint saved to {ckpt_path}")

```

```

Epoch 001 Train: 100%|          175/175 [00:15<00:00, 11.56it/s]
Epoch 001 Val: 100%|          765/765 [00:36<00:00, 20.84it/s]
Collecting any pairs: 14%|          1/7 [00:00<00:00, 6393.76it/s]
Val metrics (all): 100%|          1/1 [00:00<00:00, 3.39it/s]
Collecting transition pairs: 14%|          1/7 [00:00<00:00, 7345.54it/s]
Val metrics (transition): 100%|          1/1 [00:00<00:00, 3.50it/s]
Collecting non_transition pairs: 14%|          1/7 [00:00<00:00, 7157.52it/s]
Val metrics (non-transition): 100%|          1/1 [00:00<00:00, 3.84it/s]

[Epoch 001] train 0.2034 | val 0.0961 | inv 0.1990 | SSIM 0.1008 | SSIM_tr

```

0.1326 | SSIM_non 0.0945 | LPIPS 0.8493 | LPIPS_tr 0.7887 | LPIPS_non 0.8187
Saved best to /mnt/c/Users/ioann/OneDrive/Documents/Projects/thesis/diffusion/em
bryo_diffusion_models/nextframe_ctxk7_128px.pt

--- Generating samples for epoch 1 ---

Saving samples (mixed): 100%| 1/1 [00:00<00:00, 1.88it/s]
Saving samples (transition): 100%| 1/1 [00:00<00:00, 2.00it/s]
Rollout sampling (DA684-4 FO): 100%| 7/7 [00:01<00:00, 6.46it/s]
Epoch 001 sample outputs: 100%| 1/1 [00:02<00:00, 2.73s/it]

Samples saved to /mnt/c/Users/ioann/OneDrive/Documents/Projects/thesis/diffusion
/diffusion_samples/epoch_001

Epoch 002 Train: 100%| 175/175 [00:12<00:00, 13.68it/s]
Epoch 002 Val: 100%| 765/765 [00:35<00:00, 21.59it/s]
Collecting any pairs: 14%| 1/7 [00:00<00:00, 6472.69it/s]
Val metrics (all): 100%| 1/1 [00:00<00:00, 3.74it/s]
Collecting transition pairs: 14%| 1/7 [00:00<00:00, 9177.91it/s]
Val metrics (transition): 100%| 1/1 [00:00<00:00, 3.75it/s]
Collecting non_transition pairs: 14%| 1/7 [00:00<00:00, 6403.52it/s]
Val metrics (non-transition): 100%| 1/1 [00:00<00:00, 3.80it/s]

[Epoch 002] train 0.0745 | val 0.0733 | inv 0.0816 | SSIM 0.0772 | SSIM_tr
0.1754 | SSIM_non 0.0637 | LPIPS 0.8797 | LPIPS_tr 0.7889 | LPIPS_non 0.9646
Saved best to /mnt/c/Users/ioann/OneDrive/Documents/Projects/thesis/diffusion/em
bryo_diffusion_models/nextframe_ctxk7_128px.pt

--- Generating samples for epoch 2 ---

Saving samples (mixed): 100%| 1/1 [00:00<00:00, 1.47it/s]
Saving samples (transition): 100%| 1/1 [00:00<00:00, 2.03it/s]
Rollout sampling (DA684-4 FO): 100%| 7/7 [00:00<00:00, 7.31it/s]
Epoch 002 sample outputs: 100%| 1/1 [00:02<00:00, 2.72s/it]

Samples saved to /mnt/c/Users/ioann/OneDrive/Documents/Projects/thesis/diffusion
/diffusion_samples/epoch_002

Epoch 003 Train: 100%| 175/175 [00:11<00:00, 15.24it/s]
Epoch 003 Val: 100%| 765/765 [00:35<00:00, 21.70it/s]
Collecting any pairs: 14%| 1/7 [00:00<00:00, 7397.36it/s]
Val metrics (all): 100%| 1/1 [00:00<00:00, 3.84it/s]
Collecting transition pairs: 14%| 1/7 [00:00<00:00, 9320.68it/s]
Val metrics (transition): 100%| 1/1 [00:00<00:00, 3.88it/s]
Collecting non_transition pairs: 14%| 1/7 [00:00<00:00, 9845.78it/s]
Val metrics (non-transition): 100%| 1/1 [00:00<00:00, 3.82it/s]

[Epoch 003] train 0.0642 | val 0.0605 | inv 0.0718 | SSIM 0.0528 | SSIM_tr
0.0877 | SSIM_non 0.0506 | LPIPS 1.0316 | LPIPS_tr 0.9752 | LPIPS_non 1.0002
Saved best to /mnt/c/Users/ioann/OneDrive/Documents/Projects/thesis/diffusion/em

```
bryo_diffusion_models/nextframe_ctxk7_128px.pt
```

```
--- Generating samples for epoch 3 ---
```

```
Saving samples (mixed): 100%|      1/1 [00:00<00:00,  2.09it/s]
Saving samples (transition): 100%|    1/1 [00:00<00:00,  2.02it/s]
Rollout sampling (DA684-4 F0): 100%|    7/7 [00:00<00:00,  7.08it/s]
Epoch 003 sample outputs: 100%|    1/1 [00:02<00:00,  2.73s/it]

Samples saved to /mnt/c/Users/ioann/OneDrive/Documents/Projects/thesis/diffusion
/diffusion_samples/epoch_003
```

```
Epoch 004 Train: 100%|    175/175 [00:11<00:00, 15.18it/s]
Epoch 004 Val: 100%|    765/765 [00:35<00:00, 21.69it/s]
Collecting any pairs: 14%|      1/7 [00:00<00:00, 8322.03it/s]
Val metrics (all): 100%|    1/1 [00:00<00:00,  3.97it/s]
Collecting transition pairs: 14%|    1/7 [00:00<00:00, 10082.46it/s]
Val metrics (transition): 100%|    1/1 [00:00<00:00,  3.99it/s]
Collecting non_transition pairs: 14%|    1/7 [00:00<00:00, 9986.44it/s]
Val metrics (non-transition): 100%|    1/1 [00:00<00:00,  4.02it/s]

[Epoch 004] train 0.0529 | val 0.0542 | inv 0.0650 | SSIM 0.1100 | SSIM_tr
0.1373 | SSIM_non 0.1402 | LPIPS 0.8889 | LPIPS_tr 0.8840 | LPIPS_non 0.7484
Saved best to /mnt/c/Users/ioann/OneDrive/Documents/Projects/thesis/diffusion/em
bryo_diffusion_models/nextframe_ctxk7_128px.pt
```

```
--- Generating samples for epoch 4 ---
```

```
Saving samples (mixed): 100%|      1/1 [00:00<00:00,  2.14it/s]
Saving samples (transition): 100%|    1/1 [00:00<00:00,  2.11it/s]
Rollout sampling (DA684-4 F0): 100%|    7/7 [00:00<00:00,  7.33it/s]
Epoch 004 sample outputs: 100%|    1/1 [00:02<00:00,  2.45s/it]

Samples saved to /mnt/c/Users/ioann/OneDrive/Documents/Projects/thesis/diffusion
/diffusion_samples/epoch_004
```

```
Training complete! Best checkpoint saved to /mnt/c/Users/ioann/OneDrive/Document
s/Projects/thesis/diffusion/embryo_diffusion_models/nextframe_ctxk7_128px.pt
```

```
[13]: # Plot training curves
if metric_history:
    epochs = np.arange(1, len(metric_history) + 1)
    train_loss = np.array([m['train_loss'] for m in metric_history],
dtype=float)
    val_loss = np.array([m['val_loss'] for m in metric_history], dtype=float)
    val_ssim = np.array([m['val_ssim'] if m['val_ssim'] is not None else np.nan
for m in metric_history], dtype=float)
```

```

    val_ssim_tr = np.array([m.get('val_ssim_tr') if m.get('val_ssim_tr') is not
↪None else np.nan for m in metric_history], dtype=float)
    val_ssim_non = np.array([m.get('val_ssim_non') if m.get('val_ssim_non') is
↪not None else np.nan for m in metric_history], dtype=float)
    val_lpips = np.array([m['val_lpips'] if m['val_lpips'] is not None else np.
↪nan for m in metric_history], dtype=float)
    val_lpips_tr = np.array([m.get('val_lpips_tr') if m.get('val_lpips_tr') is
↪not None else np.nan for m in metric_history], dtype=float)
    val_lpips_non = np.array([m.get('val_lpips_non') if m.get('val_lpips_non')
↪is not None else np.nan for m in metric_history], dtype=float)

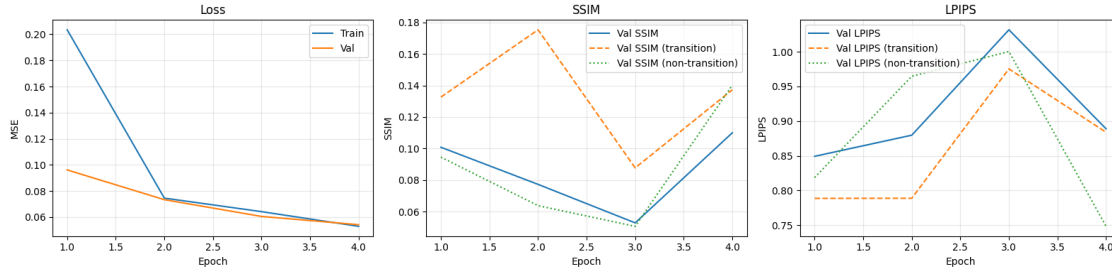
    fig, axes = plt.subplots(1, 3, figsize=(16, 4))
    axes[0].plot(epochs, train_loss, label='Train')
    axes[0].plot(epochs, val_loss, label='Val')
    axes[0].set_title('Loss')
    axes[0].set_xlabel('Epoch')
    axes[0].set_ylabel('MSE')
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)

    axes[1].plot(epochs, val_ssim, label='Val SSIM')
    axes[1].plot(epochs, val_ssim_tr, label='Val SSIM (transition)',
↪linestyle='--')
    axes[1].plot(epochs, val_ssim_non, label='Val SSIM (non-transition)',
↪linestyle=':')
    axes[1].set_title('SSIM')
    axes[1].set_xlabel('Epoch')
    axes[1].set_ylabel('SSIM')
    axes[1].grid(True, alpha=0.3)
    axes[1].legend()

    axes[2].plot(epochs, val_lpips, label='Val LPIPS')
    axes[2].plot(epochs, val_lpips_tr, label='Val LPIPS (transition)',
↪linestyle='--')
    axes[2].plot(epochs, val_lpips_non, label='Val LPIPS (non-transition)',
↪linestyle=':')
    axes[2].set_title('LPIPS')
    axes[2].set_xlabel('Epoch')
    axes[2].set_ylabel('LPIPS')
    axes[2].grid(True, alpha=0.3)
    axes[2].legend()

    plt.tight_layout()
    plt.show()
else:
    print('No metrics logged yet.')

```



```
[14]: def _stack_context_paths(ctx_paths: list[str]) -> torch.Tensor:
    assert len(ctx_paths) == CONTEXT_K, f"Expected {CONTEXT_K} context paths"
    ctx_tensors = [load_image(p).squeeze(0) for p in ctx_paths] # [k, H, W]
    return torch.stack(ctx_tensors, dim=0).unsqueeze(0) # [1, k, H, W]

def _infer_frame_index(path: str) -> int | None:
    base = os.path.basename(path)
    m = re.search(r"(\d+)\.jpe?g$", base)
    return int(m.group(1)) if m else None

def _infer_time_scalar(context_paths: list[str], max_frame_index: int | None = None) -> float:
    # Use last context frame index as t_end; normalize by max_frame_index if provided
    t_end = _infer_frame_index(context_paths[-1])
    if t_end is None:
        return 0.0
    if max_frame_index is None or max_frame_index <= 0:
        max_frame_index = t_end
    return float(t_end) / float(max(1, max_frame_index))

def visualize_samples(num=4, plane='F0', use_classifier_guidance: bool = True, save_dir: str | None = None):
    """Save the same sample grids as training: mixed, transition, and rollout grids."""
    clf = phase_classifier if (use_classifier_guidance and phase_classifier is not None and USE_CLASSIFIER_GUIDANCE) else None
    if save_dir is None:
        save_dir = OUT_SAMPLES_DIR / 'ad_hoc_samples' / plane
    save_dir = Path(save_dir)
    save_dir.mkdir(parents=True, exist_ok=True)

    # Mixed samples
    samples = val_dataset.sample_pairs_for_plane(plane, num)
    if samples:
```

```

        _save_sample_grid(samples, save_dir, title_suffix='(mixed)',
        ↪ classifier_for_infer=clf)

    # Transitional-only samples
    trans_dir = save_dir / 'transitions'
    trans_samples = val_dataset.sample_transition_pairs_for_plane(plane, num)
    if trans_samples:
        _save_sample_grid(trans_samples, trans_dir,
        ↪ title_suffix='(transition)', classifier_for_infer=clf)

    # Rollout grid (single seed)
    rollout_dir = save_dir / 'rollouts'
    rollout_seed = val_dataset.sample_pairs_for_plane(plane, 1)
    if rollout_seed:
        _save_rollout_grid(rollout_seed[0], rollout_dir, steps=CONTEXT_K,
        ↪ classifier_for_infer=clf)

    print(f"Ad hoc samples saved to {save_dir}")

def predict_next_frame(context_paths: list[str], out_path: str,
    ↪ use_classifier_guidance: bool = True, max_frame_index: int | None = None):
    """Generate the next frame given k context frames."""
    assert len(context_paths) == CONTEXT_K, f"Need exactly {CONTEXT_K} context
    ↪ frames"
    clf = phase_classifier if (use_classifier_guidance and phase_classifier is
    ↪ not None and USE_CLASSIFIER_GUIDANCE) else None
    x_ctx = _stack_context_paths(context_paths).to(device)
    frame_idx = None
    if USE_TIME_MAP:
        t_end = _infer_frame_index(context_paths[-1]) or 0
        frame_idx = min(int(t_end), int(MAX_FRAME_INDEX))
        frame_idx = torch.tensor([frame_idx], device=device, dtype=torch.long)
    x_gen = sample_from_model(
        (1, 1, IMG_SIZE, IMG_SIZE),
        x_ctx=x_ctx,
        frame_idx=frame_idx,
        classifier=clf,
        cls=None,
    )
    out_path = str(out_path)
    save_tensor_image(x_gen[0], out_path)
    return out_path

def rollout_next_frames(
    initial_context_paths: list[str],
    steps: int,

```

```

    out_dir: str,
    use_classifier_guidance: bool = True,
    max_frame_index: int | None = None
):
    """Multi-step rollout with a moving context window."""
    assert len(initial_context_paths) == CONTEXT_K, f"Need exactly {CONTEXT_K} context frames"
    os.makedirs(out_dir, exist_ok=True)
    clf = phase_classifier if (use_classifier_guidance and phase_classifier is not None and USE_CLASSIFIER_GUIDANCE) else None

    # initialize context tensors (each [1, H, W]) in [-1,1]
    ctx_tensors = [load_image(p) for p in initial_context_paths]
    t_end = _infer_frame_index(initial_context_paths[-1]) or 0

    generated_paths = []
    frame_idx = min(int(t_end), int(MAX_FRAME_INDEX))
    for step in range(1, steps + 1):
        x_ctx = torch.stack([t.squeeze(0) for t in ctx_tensors], dim=0).
        ↪unsqueeze(0).to(device)
        if USE_TIME_MAP:
            frame_idx = min(frame_idx, int(MAX_FRAME_INDEX))
            frame_idx_tensor = torch.tensor([frame_idx], device=device,
        ↪dtype=torch.long)
        else:
            frame_idx_tensor = None
        x_gen = sample_from_model(
            (1, 1, IMG_SIZE, IMG_SIZE),
            x_ctx=x_ctx,
            frame_idx=frame_idx_tensor,
            classifier=clf,
            cls=None,
        )
        save_path = os.path.join(out_dir, f'gen_t{t_end+1:05d}.png')
        save_tensor_image(x_gen[0], save_path)
        generated_paths.append(save_path)

    # update context window: drop oldest, append generated
    ctx_tensors = ctx_tensors[1:] + [x_gen[0].detach().cpu()]
    t_end += 1
    frame_idx += 1

    return generated_paths

visualize_samples(use_classifier_guidance=False)

```

Saving samples (mixed): 100%| 4/4 [00:02<00:00, 1.77it/s]

Saving samples (transition): 100%| 4/4 [00:02<00:00, 1.90it/s]
Rollout sampling (DA684-4 F0): 100%| 7/7 [00:00<00:00, 7.97it/s]
Ad hoc samples saved to /mnt/c/Users/ioann/OneDrive/Documents/Projects/thesis/di
ffusion/diffusion_samples/ad_hoc_samples/F0